

Izveštaj ispitnog projekta

Sistemi za rad u realnom vremenu - Igra pamćenja

Nikola Popović 2022201730, Tehnički fakultet - Univerzitet Singidunum

Igra pamćenja na mikrokontroleru Arduino UNO. Igra prikazuje sekvencu svetlosnih signala koje igrač mora da ponovi pritiskom na odgovarajuće tastere. Igrač ima tri pokušaja da uspešno unese sekvencu. Sa svakim uspešno završenim nivoom sekvenca se produžuje za jedan element i ubrzava prikaz.

- Za realizaciju ispitnog projekta korišćen je Arduino UNO mikrokontroler
- Programski jezik C++, Arduino Framework
- 4 tastera, pinovi 2-5
- 7 LED dioda (pinovi 6-12)
- Veribord
- Mašina stanja (6 stanja)

Svi tasteri koriste **INPUT_PULLUP** otpornik što znači da je pin normalno u stanju HIGH. Iz tog razloga spoljašni prekidi su konfigurisani na falling edge. Svaka LED dioda povezana je kroz odgovarajući otpornik 220 Ω između digitalnog pina i GND.

Idejno rešenje

Sistem je organizovan kao mašina stanja sa 6 stanja:

```
enum GameState {  
    STATE_GENERATE,  
    STATE_SHOW,  
    STATE_INPUT,  
    STATE_SUCCESS,  
    STATE_WRONG,  
    STATE_GAME_OVER  
};
```

Svaki poziv funkcije loop() odredi jedno stanje i eventualno postavi novo.

Vremenski timing se potpuno oslanja na hardverski Timer1 a detekcija tastera isključivo na prekide.

Stanja Igre:

STATE_GENERATE	Generiše novu sekvencu, resetuje level, tries i velocity
STATE_SHOW	Prikazuje sekvencu LED-a igraču
STATE_INPUT	Prima unos igrača i poredi sa sekvencom
STATE_SUCCESS	Prikaz uspeha, povećava level, ubrzava i resetuje tries.
STATE_WRONG	Beleži grešku, trepće crvenom LED inkrementuje tries
STATE_GAME_OVER	Solid crvena LED, čeka pritisak na restart

GENERATE (nova igra)

→ SHOW(sekvenca)

→ INPUT

→ SUCCESS(tačan unos) → SHOW (sledeći level level++, velocity -= 50, tries = 0, sekvenca)

→ WRONG(pogrešan unos, tries ++) → SHOW (pokušaj ponovo)

(if tries == 3) → GAME_OVER → GENERATE(restart)

Opis koda

Mapiranje pinova:

```
const int taster_1 = 2;
const int taster_2 = 3;
const int taster_3 = 4;
const int taster_4 = 5;

const int LED_y      = 6;
const int LED_r      = 7;
const int LED_succes = 8;
const int LED_1      = 9;
const int LED_2      = 10;
const int LED_3      = 11;
const int LED_4      = 12;
```

Mašina stanja:

```
enum GameState {
    STATE_GENERATE,
    STATE_SHOW,
    STATE_INPUT,
    STATE_SUCCESS,
    STATE_WRONG,
    STATE_GAME_OVER
};
```

Globalne promenljive:

sequence[]: Generisana sekvenca LED-eva koju igrač mora zapamtiti.

input_index: Gde se igrač trenutno nalazi u sekvenci dok unosi.

state: Trenutno stanje State machine-a.

```
const int MAX_LEVEL = 100;
const int MAX_TRIES = 3;
int sequence[MAX_LEVEL];
int level      = 1;
int tries      = 0;
int input_index = 0;
int velocity    = 1000;
GameState state = STATE_GENERATE;
```

Promenljive koje se menjaju unutar ISR-a (Interrupt Service Routine), a čitaju van njega, moraju biti deklarirane kao **volatile**. Bez ove ključne reči kompajler bi mogao da ih kešira u registru procesora i nikada ne bi video izmenu koju ISR napravi u pozadini.

```
volatile bool      timer_fired      = false;
volatile unsigned int timer_ms_remaining = 0;
volatile int button_pressed = -1;
```

ISR Timer1, softverski odbrojač, dekrementuje promenljivu timer_ms_remaining jednom po milisekundi. Kada dostigne nulu postavlja flag timer_fired = true.

```
ISR(TIMER1_COMPA_vect) {
    if (timer_ms_remaining > 0) {
        timer_ms_remaining--;
    } else {
        timer_fired = true;
    }
}
```

Konfiguracija Timer1, vremenski prekid:

Timer1 je 16bit hardverski timer na ATmega328P koji se koristi u CTC modu (Clear Timer Compare). U ovom modu timer broji od 0 do vrednosti upisane u registar OCR1A zatim se resetuje i okida prekid. Prescaler 64 deli sistemski clock sa 16MHz na 250KHz sto uz OCR1A = 249 daje tačno 1ms period okidanja.

```
void timer1_init() {
    TCCR1A = 0;
    TCCR1B = (1 << WGM12) | (1 << CS11) | (1 << CS10); // CTC, prescaler 64
    OCR1A = 249; // 1ms at 16MHz
    TIMSK1 = (1 << OCIE1A);
    sei();
}
```

Funkcija timer_wait(ms), postavlja odbrojač i blokira u petlji dok ISR ne postavi flag.

timer_ms_remaining = ms-1: ISR prvo dekrementuje pa proverava.

```
void timer_wait(unsigned int ms) {
    timer_fired = false;
    timer_ms_remaining = ms - 1;
    while (!timer_fired);
}
```

Spoljašni prekidi za tastere:

ATmega328P ima dva external interrupt pin-a:

- **pin2 (INT0)**
- **pin3 (INT1)**

pinovi 4 i 5 ne podržavaju ovaj tip prekida pa se za njih koristi **Pin Change Interrupt (PCINT)** koji detektuje bilo kakvu promenu na grupi pinova.

PCINT2 okida se na bilo koju promenu na PORTD pinovima 4 i 5 (HIGH LOW ili LOW HIGH).
Ručna provera koji pin je zapravo LOW.

if (button_pressed != -1) u svakom ISR sprečava da novi pritisak pregazi predhodni koji još nije obradjen (mehanizam zaštite od višestrukih okidanja).

```
ISR(INT0_vect) { if (button_pressed == -1) button_pressed = LED_1; } // pin 2
ISR(INT1_vect) { if (button_pressed == -1) button_pressed = LED_2; } // pin 3

ISR(PCINT2_vect) { // PORTD covers pins 0-7, handles pin 4 and 5
  if (button_pressed != -1) return;
  if (digitalRead(taster_3) == LOW) button_pressed = LED_3; // pin 4
  else if (digitalRead(taster_4) == LOW) button_pressed = LED_4; // pin 5
}
```

Inicijalizacija prekida:

EICRA (External Interrupt Control Register A).

Svaki interrupt ima dva bit-a ISCx1 i ISCx0 koji određuju na sta reguje:

ISCx1	ISCx0	okida se na
0	0	LOW level
0	1	bilo koju promenu
1	0	falling edge (HIGH LOW)
1	1	rising edge (LOW HIGH)

Koristimo **falling edge** jer su tasteri na INPUT_PULLUP.

EIMSK: External Interrupt Mask Register aktivira INT0 i INT1. Bez ovoga EICRA podešavanje nema efekta.

PCICR |= (1<<PCIE2); PCINT za PORTD grupu pinovi (0-7).

PCMSK2 |= (1<<PCINT20) | (1<<PCINT21): Aktiviraj samo pin4 (PCINT20) i pin5 (PCINT21).

```
void interrupts_init() {
    // INT0 (pin 2) and INT1 (pin 3) – falling edge
    EICRA = (1 << ISC01) | (1 << ISC11);
    EIMSK = (1 << INT0) | (1 << INT1);

    // Pin-Change on PORTD for pins 4 (PCINT20) and 5 (PCINT21)
    PCICR |= (1 << PCIE2);
    PCMSK2 |= (1 << PCINT20) | (1 << PCINT21);
}
```

timer1_init() → Timer1 kuca svakih 1ms → okida ISR(TIMER1_COMPA_vect)

→ dekrementuj ms_remainig ili postavi timer_fired = true

→ timer_wait() izlazi iz while petlje

interrupts_init() → pin 2/3 → INT0/INT1 → ISR odmah zna koji pin

pin 4/5 → PCINT2 → ISR mora sam da proveri digitalRead()

Pomoćne funkcije:

```
void all_leds_off() {
    digitalWrite(LED_1, LOW);
    digitalWrite(LED_2, LOW);
    digitalWrite(LED_3, LOW);
    digitalWrite(LED_4, LOW);
    digitalWrite(LED_y, LOW);
    digitalWrite(LED_r, LOW);
    digitalWrite(LED_succes, LOW);
}

void blink_red(int times) {
    for (int i = 0; i < times; i++) {
        digitalWrite(LED_r, HIGH);
        timer_wait(250);
        digitalWrite(LED_r, LOW);
        timer_wait(250);
    }
}
```

Logika igre

Generisanje sekvence:

Cela sekvenca (100 elemenata) se generiše unapred jednom na početku igre. Niz led_map[] je neophodan jer random(0,4) vraća 0-3 a LED konstante su 9-12 pin-ovi.

```
void generate_sequence() {
    randomSeed(millis());
    int led_map[4] = {LED_1, LED_2, LED_3, LED_4};
    for (int i = 0; i < MAX_LEVEL; i++) {
        sequence[i] = led_map[random(0, 4)];
    }
}
```

Prikaz sekvence:

```
void show_sequence() {
    all_leds_off();
    timer_wait(500);

    for (int i = 0; i < level; i++) {
        digitalWrite(sequence[i], HIGH);
        timer_wait(velocity);
        digitalWrite(sequence[i], LOW);
        timer_wait(200);
    }
}
```

Unos igrača

Debounce: mehanički tasteri pri pritisku mogu okidati interrupt više puta u par milisekundi. Čekanje od 200ms (timer_wait(200)) obezbedjuje da se lažni signali ignorišu.

```
int wait_for_button() {
    button_pressed = -1;
    while (button_pressed == -1); // ISR sets this
    int pressed = button_pressed;
    timer_wait(200); // debounce
    button_pressed = -1;
    return pressed;
}
```

Setup:

```
void setup() {  
    pinMode(taster_1, INPUT_PULLUP);  
    pinMode(taster_2, INPUT_PULLUP);  
    pinMode(taster_3, INPUT_PULLUP);  
    pinMode(taster_4, INPUT_PULLUP);  
  
    pinMode(LED_y,      OUTPUT);  
    pinMode(LED_r,      OUTPUT);  
    pinMode(LED_succes, OUTPUT);  
    pinMode(LED_1,      OUTPUT);  
    pinMode(LED_2,      OUTPUT);  
    pinMode(LED_3,      OUTPUT);  
    pinMode(LED_4,      OUTPUT);  
  
    timer1_init();  
    interrupts_init();  
  
    state = STATE_GENERATE;  
}
```

Igra:

Broj treptanja crvene LED diode direktno odgovara broju grešaka. Na trećoj grešci prelazi se u STATE_GAME_OVER gde crvena LED dioda osatje upaljena sve dok igrač ne pritisne bilo koji taster za restart.

Žuta dioda svetli tokom unosa sekvence igraca.

Kada se unese uspešan redosled sekvence sve zelene diode će se upaliti dajući indicaciju uspeha.

loop:

```
void loop() {  
    switch (state) {  
  
        case STATE_GENERATE:  
            generate_sequence();  
            level      = 2;  
            tries      = 0;  
            velocity   = 1000;  
            state      = STATE_SHOW;  
            break;
```



```

case STATE_SHOW:
    show_sequence();
    input_index = 0;
    state = STATE_INPUT;
    break;

case STATE_INPUT: {
    digitalWrite(LED_y, HIGH);

    int pressed = wait_for_button();

    digitalWrite(pressed, HIGH);
    timer_wait(300);
    digitalWrite(pressed, LOW);

    if (pressed == sequence[input_index]) {
        input_index++;
        if (input_index == level) {
            digitalWrite(LED_y, LOW);
            state = STATE_SUCCESS;
        }
    } else {
        digitalWrite(LED_y, LOW);
        state = STATE_WRONG;
    }
    break;
}

case STATE_SUCCESS:
    tries = 0;
    all_leds_off();

    digitalWrite(LED_succes, HIGH);
    digitalWrite(LED_1, HIGH);
    digitalWrite(LED_2, HIGH);
    digitalWrite(LED_3, HIGH);
    digitalWrite(LED_4, HIGH);
    timer_wait(1000);
    all_leds_off();

    if (level < MAX_LEVEL) level++;
    if (velocity > 100)    velocity -= 50;

    state = STATE_SHOW;

```

```

        break;

    case STATE_WRONG:
        tries++;
        blink_red(tries);

        if (tries >= MAX_TRIES) {
            state = STATE_GAME_OVER;
        } else {
            input_index = 0;
            state = STATE_SHOW; // try again
        }
        break;

    case STATE_GAME_OVER:
        all_leds_off();
        digitalWrite(LED_r, HIGH);

        wait_for_button();

        digitalWrite(LED_r, LOW);
        state = STATE_GENERATE;
        break;
    }
}

```

Zaključak

Projekat uspešno implementira igru na Arduino Uno platformi uz poštovanje svih postavljenih zahteva.

Mašina stanja sa šest jasno definisanih stanja. Vremenski prekid (Timer1 CTC, period okidanja 1ms) koristi se za sva merenja vremena u sistemu, prikaz sekvence debounce i animacije bez upotrebe funkcije delay(). Spoljasni prekidi (INT0, INT1, PCINT2) detektuju sve pritiske tastera bez poolinga u glavnoj petlji.